Computer Programming

Unit 3:

Parsers

Contents

LR(1) Grammar

- SLR is so simple and can only represent the small group of grammar.
- LR(1) parsing uses **look ahead** to **avoid unnecessary conflicts** in **parsing table**.

LR(1) item = LR(0) item + look-ahead

LR(0) item	LR(1) item
$[A \rightarrow \alpha \bullet \beta]$	$[A \rightarrow \alpha \bullet \beta, a]$

Constructing LR(1) Parsing Table

- SLR used the LR(0) items

  i.e. items used were productions with an embedded dot,

  but contained no other (lookahead) information.

- The LR(1) items contain the same productions with embedded dots

  but added a **second component**.

This second component becomes important only when the dot is at the **extreme right.**

For LR(1) we do that reduction only if the input symbol is exactly the second component of the item.

- When Constructing a parsing table for **action and goto**
  involves **building a state machine** that can **identify valid handles**

- For **building a state machine**, we need to define **three terms:**
  - closure operations
  - goto operations
  - Canonical collection of LR(1) item

Computation of **Closure for LR(1)Items**

The closure of a set S of LR(1) items for augmented grammar G is computed as follows,

1. Initialize closure(I) = I

   (where I is a set of LR(1) items)

2. If item **[A→α•Bβ, a]**

   and $\boldsymbol{B \rightarrow \gamma}$ is a production rule,

   then add **[B→•γ, b]** in closure(I)

   where **b ∈ FIRST(βa)**

(B is a non-terminal. α, β, γ are any string which can be terminal, non-terminal)

3. Repeat 2 until no new items can be added.

Computation of **Goto Operation for LR(1) Items**

If I is a set of LR(1) items

and X is a symbol (terminal or non-terminal),

then goto(I, X) is computed as :

1. If item **[A→α•Xβ, a]** in I,

   then add items **closure({[A→αX•β, a]})** to goto(I,X)

2. Repeat until no more items can be added to goto(I, X)

Construction of **Canonical LR(1) Collection**

- Is a **collection** of **sets of LR(1) item sets (states)**
  used to build an LR(1) parsing table.

i.e. It is the set:

$I_0, I_1, I_2, \ldots, I_n$

- To construct canonical LR(1) collection for a grammar,
  we require
  augmented grammar,
  closure
  and goto functions.

Algorithm

1. **Augment the grammar**

   Add new start symbol:

   S‘ → S

2. **Construct Start state**

   I0 = closure({S‘→•S, $})

   where, $ is look-ahead symbol
   -which one to choose

3. **Generate states using GOTO**

   For every state I and symbol X:

   Goto(I, X) → new state

Repeat until no new states appear.

Construction of LR(1) Parsing Table **Algorithm**

Algorithm

1. **Augment the Grammar**

Add a new start symbol production of form

$$S' \rightarrow S$$

2. **Construct Canonical Collection of LR(1) items**

$\{I_0, I_1, I_2, \ldots, I_n\}$

3. Construct Initial State I_0

I0 = closure({S'→•S, $})

where, $ is look-ahead symbol - looking for next symbol

4. **Create Parsing Table Format**

Split into:

ACTION table

- Columns = terminals + $
- Contains:
  - shift s
  - reduce A → α
  - accept
  - Error

GOTO table

- Columns = non-terminals
- Contains **state transitions**

5. Fill ACTION Table

a. SHIFT Rule

If $[A \rightarrow \alpha \bullet a\beta, b] \in Ii$ and goto (Ii, a) = Ij

then **action** [i, a] = shift j

where Ii - an item in state Ii

a is a terminal

b. REDUCE Rule

If $[A \rightarrow \alpha \bullet, a] \in Ii$,

then action [i, a] = **reduce** $A \rightarrow \alpha$ where $A \neq S'$.

c. ACCEPT Rule

If $[S' \rightarrow S \bullet, \$]$ is in Ii ,

then action[i, $] = **accept**

d. If any **conflicting actions** generated by these rules, the **grammar is not LR(1).**

6. Fill GOTO Table

For all non-terminals A,

if goto (Ii, A) = Ij

then goto [i, A] = j

Eg 1, Construct LR(1) parsing table of following grammar,

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

Solution:

Step 1: Augment the Grammar

1. $S' \rightarrow S$
2. $S \rightarrow AA$
3. $A \rightarrow aA$
4. $A \rightarrow b$

Step 2: Construct Canonical Collection of LR(1) items

$\{I_0, I_1, I_2, \ldots, I_n\}$

Start State I_0

I_0 = Closure ({S' → •S, $})

= { S' → •S, $

S → • AA, $

A → • aA, a|b

A → • b, a|b

}

If [A→α•Bβ, a]

and $B \rightarrow \gamma$ is a production rule,

then *closure(I) = add [B→•γ, b] in*
where b ∈ FIRST(βa)

expanding S in S ⟶ • S, $ and applying closure rule

1. lookahead = *FIRST(βa)* = FIRST($) = {$}
   β = empty, a = $
   lookahead = FIRST($) = {$}
   So, result = S ⟶ • AA, $

expanding A in S ⟶ • AA, $ and applying closure rule

1. lookahead = *FIRST(βa)* = FIRST(A$) = {a,b})

2. lookahead = *FIRST(βa)* = FIRST(A$) = {a,b})

$ - lookahead – looking for next symbol

Goto (I_0, S)

= closure (S‘ → S•, $)

= { S‘ → S•, $

}

= I1

Goto (I_0, A)

= closure (S → A•A, $)

= { S → A•A, $

A → •aA, $

A → •b, $

}

= I2

If [A→α•Xβ, a],
Goto (I,X) = closure({[A→αX•β, a]})

If [A→α•Bβ, a]
and $B \rightarrow \gamma$ is a production rule,
then closure(I) = [B→•γ, b]
where b ∈ FIRST(βa)

expanding A in S → A• A, $ and applying closure rule

1. lookahead = *FIRST(βa)* = FIRST($) = {$}

β = empty a = $

So, A → •aA, $

Same for A → •b, $

Goto (I_0, a)

= Closure ($A \rightarrow a \bullet A, a|b$)

= { $A \rightarrow a \bullet A, a|b$

$A \rightarrow \bullet aA, a|b$

$A \rightarrow \bullet b, a|b$

}

= I_3

Goto (I_0, b)

= closure ($A \rightarrow b \bullet, a|b$)

= { $A \rightarrow b \bullet, a|b$

}

= I_4

If $[A \rightarrow \alpha \bullet X\beta, a]$,

$goto(I,X) = closure(\{[A \rightarrow \alpha X \bullet \beta, a]\})$

Goto (I_2, A)

= Closure ($S \rightarrow AA \bullet$, $)

= { $S \rightarrow AA \bullet$, $

}

= I_5

Goto (I_2, a)

= Closure ($A \rightarrow a \bullet A$, $)

= { $A \rightarrow a \bullet A$, $

$A \rightarrow \bullet aA$, $

$A \rightarrow \bullet b$, $

}

= I_6

Goto (I_2, b)

= Closure (A ⟶ b•, $)

= { A ⟶ b•, $

}

= I7

Goto (I_3, A)

= Closure (A ⟶ aA•, a|b)

= { A ⟶ aA•, a|b

}

= I8

If [A⟶α•Xβ, a],

goto(I,X) = closure({[A⟶αX•β, a]})

Goto (I_3, a)

= Closure (A ⟶ a•A, a|b)

= { A ⟶ a•A, a|b

A ⟶ •aA, a|b

A ⟶ •b, a|b}

}

= I3 (same)

Goto (I3, b)

= Closure (A ⟶ b•, a|b)

= I4 (same)

Goto (I6, A)

= Closure (A → aA•, $)

= { A → aA•, $

}

= I9

Goto (I6, a)

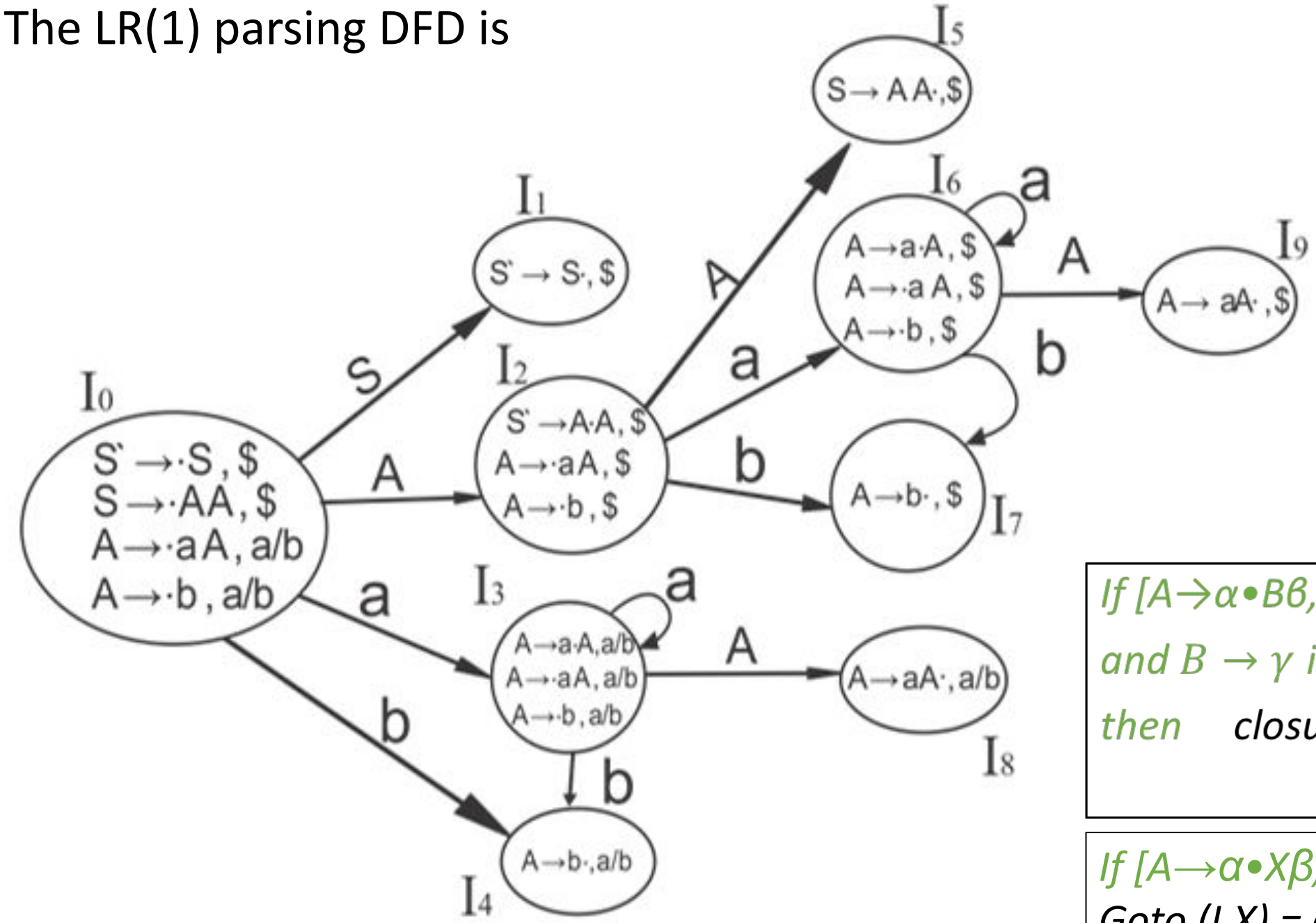
= Closure (A → a•A, $) = I6 (same)

Goto (I6, b)

= Closure (A → b•, $) = I7 (same)

If [A→α•Xβ, a],
goto(I,X) = closure({[A→αX•β, a]})

The LR(1) parsing DFD is



S' □ S
S → AA
A → aA
A → b

If [A→α•Bβ, a]

and B → γ is a production rule,

then *closure(I) = [B→•γ, b]*

where b ∈ FIRST(βa)

If [A→α•Xβ, a],

Goto (I,X) = closure({[A→αX•β, a]})

Now The LR(1) parsing table is

States (I)	Action table			Goto table	
	a	b	$	S	A
I0	s3	s4		1	2
I1			accep t		
I2	s6	s7			5
I3	s3	s4			8
I4	r4	r4			
I5			r2		
I6	s6	s7			9
I7			r4		
I8	r3	r3			
I9			r3		

If [A → α•aβ, b] shift

If [A → α•, a] **reduce A-> α**

If [S'→ S•, $] **accept**

goto- write numbers

shift – s reduce - r

1. S' → S
2. S → AA
3. A → aA
4. A → b

Eg 2, Construct canonical LR(1) collection of the following grammar

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \in$

$B \rightarrow \in$

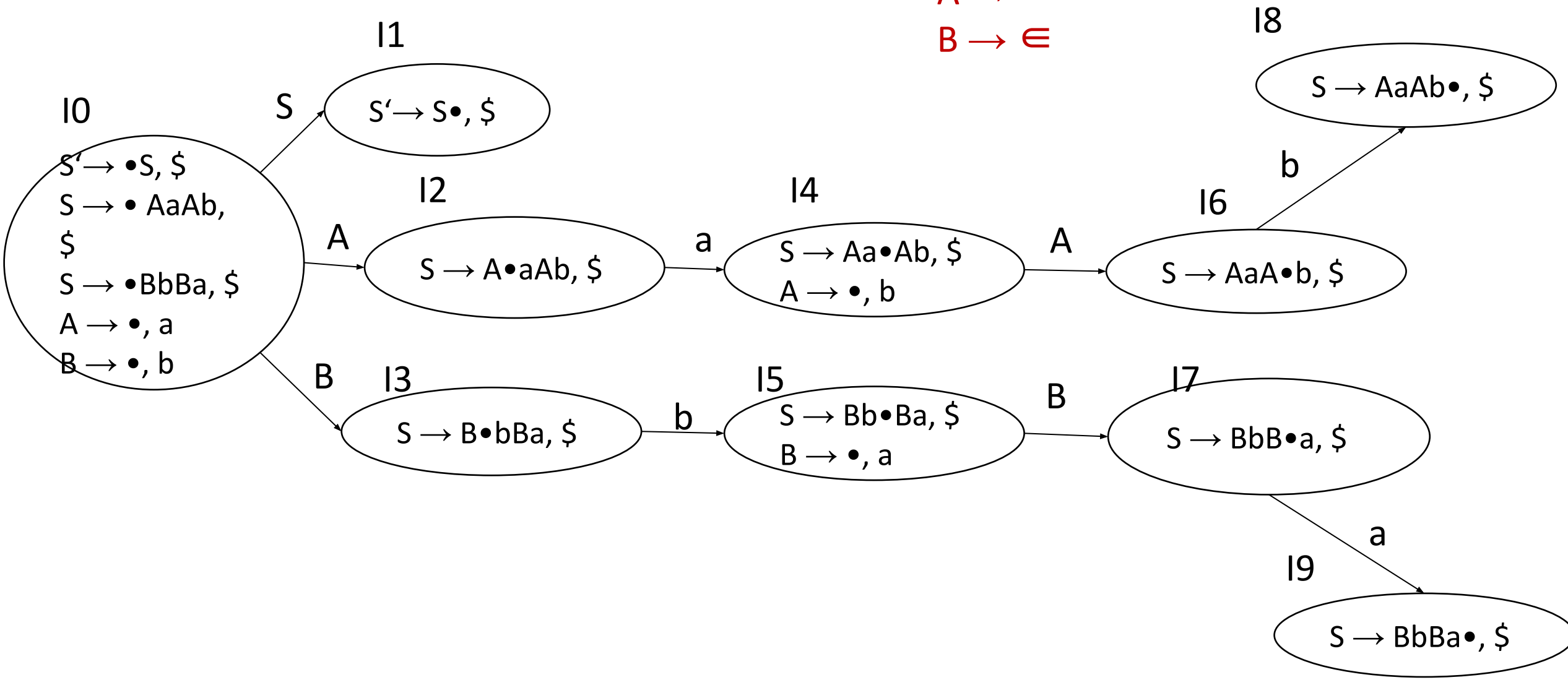
Solution:

Its Augmented Grammar is

1. $S' \rightarrow S$
2. $S \rightarrow AaAb$
3. $S \rightarrow BbBa$
4. $A \rightarrow \in$
5. $B \rightarrow \in$

The LR(1) parsing DFD is

S‘ → S
S → AaAb
S → BbBa
A → ∈
B → ∈



Now The LR(1) parsing table is

States (I)	Action table			Goto table		
	a	b	$	S	A	B
I0	r3	r4		1	2	3
I1			accep t			
I2	s4					
I3		s5				
I4					6	
I5						7
I6		s8				
I7	s9					
I8			r2			
I9			r3			

If [A → α•aβ, b] shift

If [A → α•, a] **reduce A-> α**

If [S'→ S•, $] **accept**

goto- write numbers

shift – s reduce - r

1. S' → S
2. S → AaAb
3. S → BbBa
4. A → ∈
5. B → ∈

Eg 2, Construct LR(1) parsing table for the augmented grammar,

S‘ → S

S → L = R

S → R

L → * R

L → id

R → L

Solution:

Step 1: The Grammar is already augmented

Step 2: Construct Canonical Collection of LR(1) items

$\{I_0, I_1, I_2, \ldots, I_n\}$

Start State I0

= Closure(S‘→•S, $)

= { S‘→•S, $

S → •L = R, $

S → •R, $

L → •* R , =

L → •id, =

R → •L, $

L → •* R , $

L → •id, $

}